

EDLIN-RS

A minimal Rust clone of the classic MS-DOS `EDLIN` line editor.

Overview

`edlin-rs` is a single-file, dependency-free line editor that mimics the command syntax and behavior of the original DOS `EDLIN` utility. It operates on a text buffer loaded into memory, accepts line-oriented editing commands, and writes the result back to disk with CRLF line endings (DOS-style).

Building

```
bash

rustc -O -o edlin edlin.rs
```

Usage

```
bash

./edlin [filename]
```

- If `filename` is given, it is loaded into the buffer (or created as a "New file" if it doesn't exist).
- If no filename is given, a default file named `_out_YYYYMMDDHHMM.txt` (based on the current UTC timestamp) is used.

On startup, the editor prints a banner and enters an interactive command loop, prompting with `*`.

Command Reference

Command	Syntax	Description
(number)	<code>[n]</code>	Go to and display line <code>[n]</code>
(blank)	<code><Enter></code>	Advance to and display the next line
<code>[L]</code>	<code>[n[,m]]L</code>	List lines (defaults to a window around the current line, max 23 lines)
<code>[P]</code>	<code>[n[,m]]P</code>	Page through lines 23 at a time, updating the current line

Command	Syntax	Description
I	[n]I	Insert text before line n (or at the current line); terminated by . on its own line
D	[n[,m]]D	Delete lines n through m (defaults to current line)
C	n,m,d[C cnt]	Copy lines n – m to before line d , optionally cnt times
M	n,m,dM	Move lines n – m to before line d
T	[n]T file	Transfer (merge) the contents of file into the buffer before line n
S	text	Search for text starting from the current line (case-insensitive)
R	old new	Replace first occurrence of old with new on/after the current line
W	[file]	Write (save) the buffer to file (or the current file)
E	E	Save and exit
Q	Q	Quit without saving (prompts Abort edit (Y/N)?)
?	?	Show help

Architecture

Core data structures

- Editor** — holds the in-memory line buffer (`Vec<String>`), the current file name, the 1-based current line pointer, and a **modified** flag.
- ParsedCmd** — represents a parsed command: a numeric range (**start**, **end**, **third**), the command letter, and any trailing argument text (**rest**).

Command parsing

`parse_command` implements EDLIN's flexible numeric-prefix grammar:

- n** — single line number
- n,m** — a range
- n,m,d** — a range plus a destination (used by **C** and **M**)
- A bare number with no following command is treated as the special pseudo-command **'#'** (go to line).

Command execution

`run_command` dispatches the parsed command to the corresponding `Editor` method:

- **Navigation:** `#` (go to line), blank input (advance one line)
- **Display:** `L` (list), `P` (page)
- **Editing:** `I` (insert), `D` (delete), `C` (copy), `M` (move), `R` (replace)
- **File I/O:** `T` (transfer/merge), `W` (write), `E` (save & exit), `Q` (quit)
- **Search:** `S`
- **Help:** `?`

File I/O details

- Files are read line-by-line, stripping trailing `\r\n` or `\n`.
- Files are written with explicit `\r\n` line endings to match DOS conventions.
- `load_file` reports "New file" if the file doesn't exist, or the line count if it does.
- `save_file` reports the number of lines written.

Timestamp generation

`current_timestamp` computes a `YYYYMMDDHHMM` UTC timestamp from `SystemTime` using a manual civil-date algorithm (Howard Hinnant's days-from-civil algorithm), avoiding any external date/time crate dependency.

Notable Implementation Notes

- All line numbers in the public command interface are **1-based**; internal `Vec` indexing is 0-based, with conversions handled carefully throughout.
- `insert_lines` mimics EDLIN's interactive insert mode, prompting `n:*` for each new line and terminating on a `.` line or a literal Ctrl+Z (`\x1A`).
- `move_lines` rejects moves where the destination falls inside the source range (prints `?`), preventing data corruption.
- The `R` (replace) command only supports a single space-delimited `old new` pair and replaces only the first match from the current line onward (or all matches if `global` were set, though no command currently sets it to `true`).
- `MAX_LINE_LEN` (4096) truncates overly long input lines and command arguments.

Known Limitations

- No undo support.

- `(R)` command doesn't expose EDLIN's `(?)` (confirm each replacement) or `(*)` (global) switches via the command line, despite `replace_lines` supporting a `global` flag internally.
- No support for EDLIN's `(A)/(Z)` (ASCII/end-of-file) hex display modes or `(#)` page-size customization.
- `current_timestamp` assumes UTC; original EDLIN used local time for default filenames.
- The `modified` flag is tracked but never used to warn on unsaved changes (e.g., on `(Q)`, only a generic prompt is shown regardless of buffer state).

License

Provided as-is for educational purposes.